

Pontificia Universidad Católica de Chile

EPG4001 Aprendizaje Supervisado

Material Complementario

Clase7MIAApSup.pdf

Jorge Luis Bazán
jlbazan@uc.cl

Librerías

Las librerías a utilizar para los procedimientos son las siguientes:

```
library(e1071)      # svm()
library(class)      # knn()
library(caret)      # confusionMatrix(), trainControl()
library(ISLR2)      # Bases de datos
library(ggplot2)    # Visualización
library(tidyverse)  # Manipulación de datos
library(pROC)       # Curvas ROC y AUC
library(knitr)      # Tablas
library(themis)     # SMOTE
```

Función auxiliar para calcular F1 Score y AUC a partir de una matriz de confusión y un vector de probabilidades:

```
# F1 Score desde una confusion matrix de caret
f1_from_cm <- function(cm) {
  VP <- cm$table[2,2]; FP <- cm$table[2,1]; FN <- cm$table[1,2]
  round(2*VP / (2*VP + FP + FN), 4)
}

# Función completa de métricas
metricas_full <- function(cm, pi_hat, y_bin, nombre) {
  VP <- cm$table[2,2]; FP <- cm$table[2,1]; FN <- cm$table[1,2]
  data.frame(
    Metodo      = nombre,
    Accuracy    = round(cm$overall["Accuracy"], 4),
    Sensibilidad = round(cm$byClass["Sensitivity"], 4),
    Especificidad = round(cm$byClass["Specificity"], 4),
    F1          = round(2*VP/(2*VP+FP+FN), 4),
    AUC         = round(as.numeric(
      pROC::auc(pROC::roc(y_bin, pi_hat, quiet=TRUE))), 4)
  )
}
```

1. Support Vector Machine (SVM)

La SVM busca el hiperplano que maximiza el margen entre clases. En R, la función `e1071::svm` implementa tanto la versión lineal como versiones con kernel para fronteras no lineales.

1.1. SVM Lineal: Ejemplo con datos simulados

```
set.seed(42)
n_sim <- 100
#genera 50 datos con media -1, y 50 datos con media 1
x1 <- c(rnorm(n_sim/2, mean = -1), rnorm(n_sim/2, mean = 1))
x1
```

```
#genera 50 datos con media -1, y 50 datos con media 1
x2 <- c(rnorm(n_sim/2, mean = -1), rnorm(n_sim/2, mean = 1))
x2
```

```
y <- factor(rep(c(-1, 1), each = n_sim/2))
y
```

[illegible]

```
hist(x1)
```

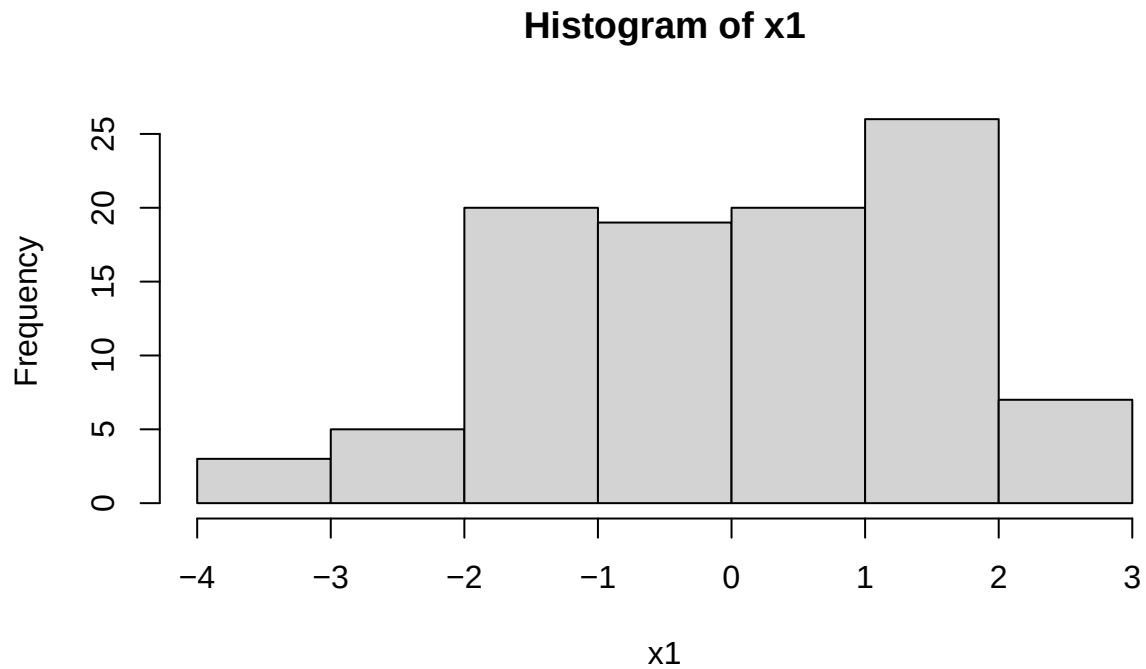


Figura 1: Datos simulados: dos clases linealmente separables

```
hist(x2)
```

```
dat_svm <- data.frame(x1 = x1, x2 = x2, y = y)
dat_svm_bin <- ifelse(dat_svm$y == "1", 1, 0)
table(dat_svm_bin)
```

```
## dat_svm_bin
##  0  1
## 50 50
```

```
data=cbind(dat_svm,dat_svm_bin)
head(data)
```

```
##           x1           x2  y dat_svm_bin
## 1  0.3709584  0.20096538 -1             0
## 2 -1.5646982  0.04475109 -1             0
## 3 -0.6368716 -2.00320865 -1             0
## 4 -0.3671374  0.84848190 -1             0
## 5 -0.5957317 -1.66677341 -1             0
## 6 -1.1061245 -0.89448619 -1             0
```

Para los datos simulados creamos una nube de puntos de X1 y X2

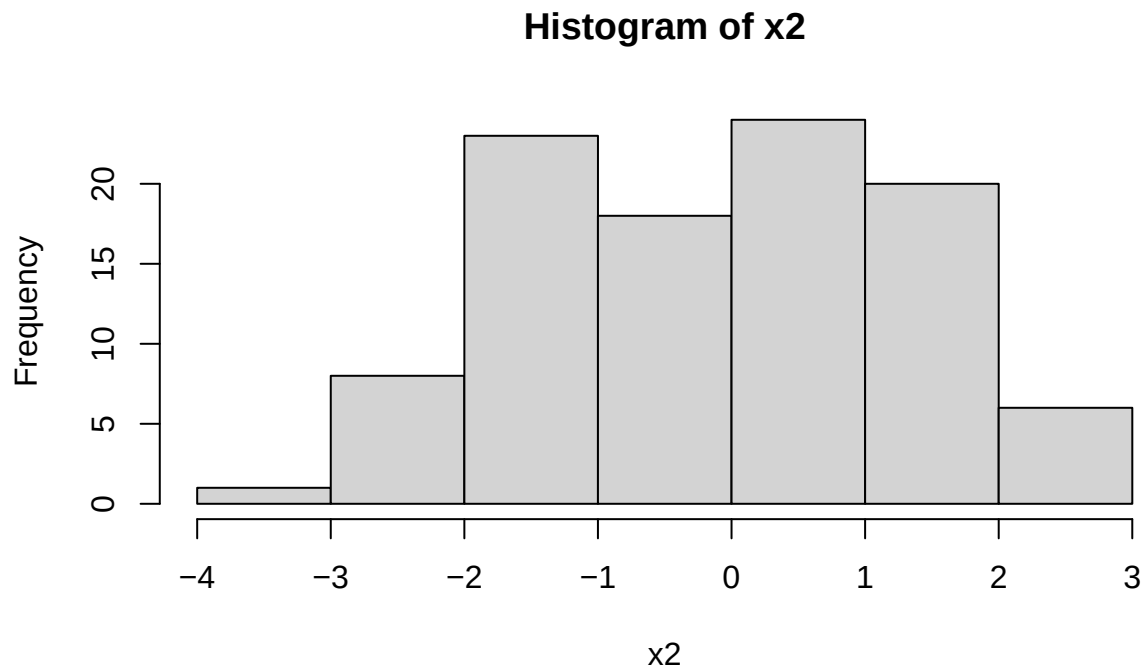
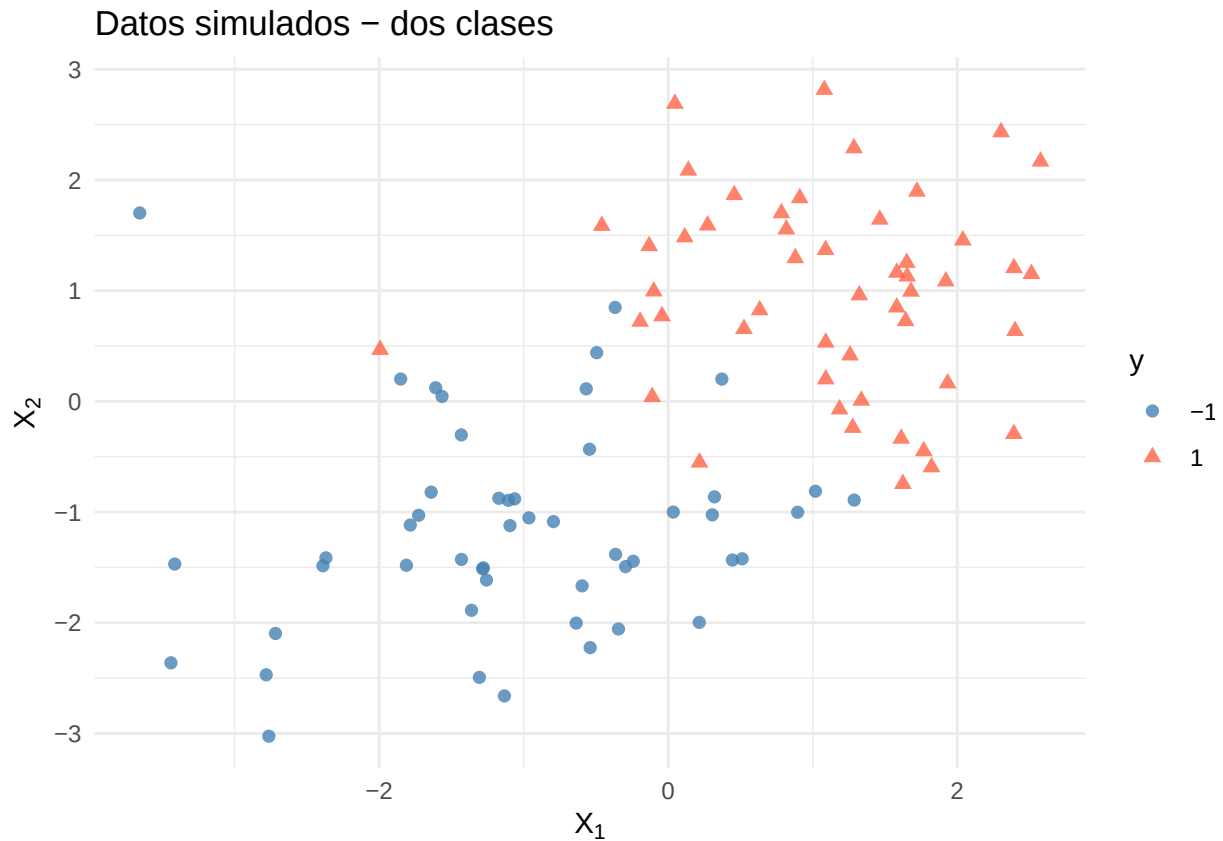


Figura 2: Datos simulados: dos clases linealmente separables

```
ggplot(dat_svm, aes(x1, x2, color = y, shape = y)) +  
  geom_point(size = 2, alpha = 0.8) +  
  scale_color_manual(values = c("steelblue", "tomato")) +  
  labs(title = "Datos simulados - dos clases",  
        x = expression(X[1]), y = expression(X[2])) +  
  theme_minimal(base_size = 11)
```



1.1.1. Ajuste con kernel lineal y metricas

```
mod_svm_lin <- svm(y ~ x1 + x2, data = dat_svm,
  kernel = "linear", cost = 1, scale = TRUE,
  probability = TRUE)
```

```
# Resumen del modelo :
summary(mod_svm_lin)
```

```
##
## Call:
## svm(formula = y ~ x1 + x2, data = dat_svm, kernel = "linear", cost = 1,
##      probability = TRUE, scale = TRUE)
##
##
## Parameters:
##   SVM-Type:  C-classification
##   SVM-Kernel: linear
##      cost:   1
##
## Number of Support Vectors:  20
##
## ( 10 10 )
##
##
```

```
## Number of Classes: 2
##
## Levels:
## -1 1
```

```
# Numero de vectores de soporte por clase :
mod_svm_lin$nSV
```

```
## [1] 10 10
```

```
pred_lin_raw <- predict(mod_svm_lin, newdata = dat_svm, probability = TRUE)
pred_lin_raw
```

```
##      1      2      3      4      5      6      7      8      9     10     11     12     13     14     15     16     17     18     19     20
##      1     -1     -1      1     -1     -1     -1     -1     -1     -1     -1     -1     -1     -1     -1     -1     -1     -1     -1     -1
##     21     22     23     24     25     26     27     28     29     30     31     32     33     34     35     36     37     38     39     40
##     -1     -1     -1     -1     -1     -1     -1     -1     -1     -1     -1     -1     -1     -1      1     -1     -1     -1     -1     -1
##     41     42     43     44     45     46     47     48     49     50     51     52     53     54     55     56     57     58     59     60
##     -1     -1     -1     -1     -1     -1     -1     -1     -1     -1      1     -1      1      1      1      1      1      1     -1      1
##     61     62     63     64     65     66     67     68     69     70     71     72     73     74     75     76     77     78     79     80
##      1      1      1      1      1      1      1      1      1      1      1      1      1      1      1      1      1      1      1      1
##     81     82     83     84     85     86     87     88     89     90     91     92     93     94     95     96     97     98     99    100
##      1      1      1      1      1      1      1      1      1      1      1      1      1      1     -1      1      1      1      1      1
## attr(,"probabilities")
##              -1              1
## 1  0.3056981004 6.943019e-01
## 2  0.8911725359 1.088275e-01
## 3  0.9919880339 8.011966e-03
## 4  0.2540659355 7.459341e-01
## 5  0.9838664485 1.613355e-02
## 6  0.9645537644 3.544624e-02
## 7  0.8945021250 1.054979e-01
## 8  0.9765706842 2.342932e-02
## 9  0.5664786117 4.335214e-01
## 10 0.9615216024 3.847840e-02
## 11 0.8384884986 1.615115e-01
## 12 0.5149508506 4.850491e-01
## 13 0.9979463502 2.053650e-03
## 14 0.9911628020 8.837198e-03
## 15 0.9988498541 1.150146e-03
## 16 0.9625048421 3.749516e-02
## 17 0.9913684388 8.631561e-03
## 18 0.8481610778 1.518389e-01
## 19 0.9999094771 9.052292e-05
## 20 0.7877680014 2.122320e-01
## 21 0.9987414197 1.258580e-03
## 22 0.9998209483 1.790517e-04
## 23 0.9662841788 3.371582e-02
## 24 0.9747505359 2.524946e-02
## 25 0.6905792316 3.094208e-01
## 26 0.9916566088 8.343391e-03
## 27 0.9926308863 7.369114e-03
## 28 0.9999373584 6.264156e-05
```

29 0.9940428086 5.957191e-03
30 0.9798219140 2.017809e-02
31 0.8385212727 1.614787e-01
32 0.9666402141 3.335979e-02
33 0.8768997422 1.231003e-01
34 0.8819571915 1.180428e-01
35 0.4721792268 5.278208e-01
36 0.9995971656 4.028344e-04
37 0.9905665849 9.433415e-03
38 0.8990020863 1.009979e-01
39 0.9994707578 5.292422e-04
40 0.9682470766 3.175292e-02
41 0.9627734658 3.722653e-02
42 0.9962227412 3.777259e-03
43 0.9608652943 3.913471e-02
44 0.9879362636 1.206374e-02
45 0.9975748286 2.425171e-03
46 0.6500861517 3.499138e-01
47 0.9954848992 4.515101e-03
48 0.9046507915 9.534921e-02
49 0.9306757026 6.932430e-02
50 0.9892911666 1.070883e-02
51 0.0271647314 9.728353e-01
52 0.7002901269 2.997099e-01
53 0.0004912605 9.995087e-01
54 0.0276261316 9.723739e-01
55 0.0804974596 9.195025e-01
56 0.2324937999 7.675062e-01
57 0.0158922671 9.841077e-01
58 0.1429549351 8.570451e-01
59 0.8658406648 1.341593e-01
60 0.0022278987 9.977721e-01
61 0.0844044538 9.155955e-01
62 0.1988445975 8.011554e-01
63 0.0130535093 9.869465e-01
64 0.0119778460 9.880222e-01
65 0.0328849338 9.671151e-01
66 0.0004251244 9.995749e-01
67 0.1479464957 8.520535e-01
68 0.0040339567 9.959660e-01
69 0.0096373396 9.903627e-01
70 0.0026404685 9.973595e-01
71 0.2036144767 7.963855e-01
72 0.0088191583 9.911808e-01
73 0.3360335251 6.639665e-01
74 0.0054461138 9.945539e-01
75 0.0153090388 9.846910e-01
76 0.0237650863 9.762349e-01
77 0.1898469447 8.101531e-01
78 0.0060820791 9.939179e-01
79 0.0492844322 9.507156e-01
80 0.1518101961 8.481898e-01
81 0.0038232788 9.961767e-01
82 0.0803395027 9.196605e-01

```
## 83 0.0170155980 9.829844e-01
## 84 0.0258491388 9.741509e-01
## 85 0.2566964509 7.433035e-01
## 86 0.1887260990 8.112739e-01
## 87 0.0135648924 9.864351e-01
## 88 0.0171421631 9.828578e-01
## 89 0.0540194787 9.459805e-01
## 90 0.2242265199 7.757735e-01
## 91 0.0040565811 9.959434e-01
## 92 0.1292465565 8.707534e-01
## 93 0.0100338107 9.899662e-01
## 94 0.0694717855 9.305282e-01
## 95 0.5355304118 4.644696e-01
## 96 0.0153359755 9.846640e-01
## 97 0.0777984755 9.222015e-01
## 98 0.0844216981 9.155783e-01
## 99 0.0010580450 9.989420e-01
## 100 0.0126734554 9.873265e-01
## Levels: -1 1
```

```
pi_lin_sim <- attr(pred_lin_raw, "probabilities")[,"1"]
pred_lin <- predict(mod_svm_lin, newdata = dat_svm)
pred_lin
```

```
## 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
## 1 -1 -1 1 -1 -1 -1 -1 -1 -1 -1 1 -1 -1 -1 -1 -1 -1 -1
## 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40
## -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 1 -1 -1 -1 -1 -1
## 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60
## -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 1 -1 1 1 1 1 1 1 -1 1
## 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80
## 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
## 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100
## 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 -1 1 1 1 1
## Levels: -1 1
```

```
#Crea Matriz de confusión
cm_lin <- confusionMatrix(pred_lin, dat_svm$y, positive = "1")
# Métricas SVM Lineal (datos simulados)
tab_lin_sim <- metricas_full(cm_lin, pi_lin_sim, dat_svm_bin, "SVM Lineal (C=1)")
#mostrando la tabla
kable(tab_lin_sim, row.names = FALSE, booktabs = TRUE,
      caption = "Métricas SVM Lineal - datos simulados (muestra de entrenamiento)")
```

Cuadro 1: Métricas SVM Lineal - datos simulados (muestra de entrenamiento)

Metodo	Accuracy	Sensibilidad	Especificidad	F1	AUC
SVM Lineal (C=1)	0.93	0.94	0.92	0.9307	0.99

```
kable(cm_lin$stable,
      caption = "Matriz de confusion - SVM Lineal (datos simulados)")
```

Cuadro 2: Matriz de confusion - SVM Lineal (datos simulados)

	-1	1
-1	46	3
1	4	47

Las métricas muestran un buen desempeño del clasificador SVM. Solamente 7 casos no fueron bien clasificados

1.1.2. Visualizacion de la frontera de decision

```
plot(mod_svm_lin, dat_svm, x2 ~ x1,
     col = c("lightblue", "lightsalmon"),
     symbolPalette = c("steelblue", "tomato"))
```

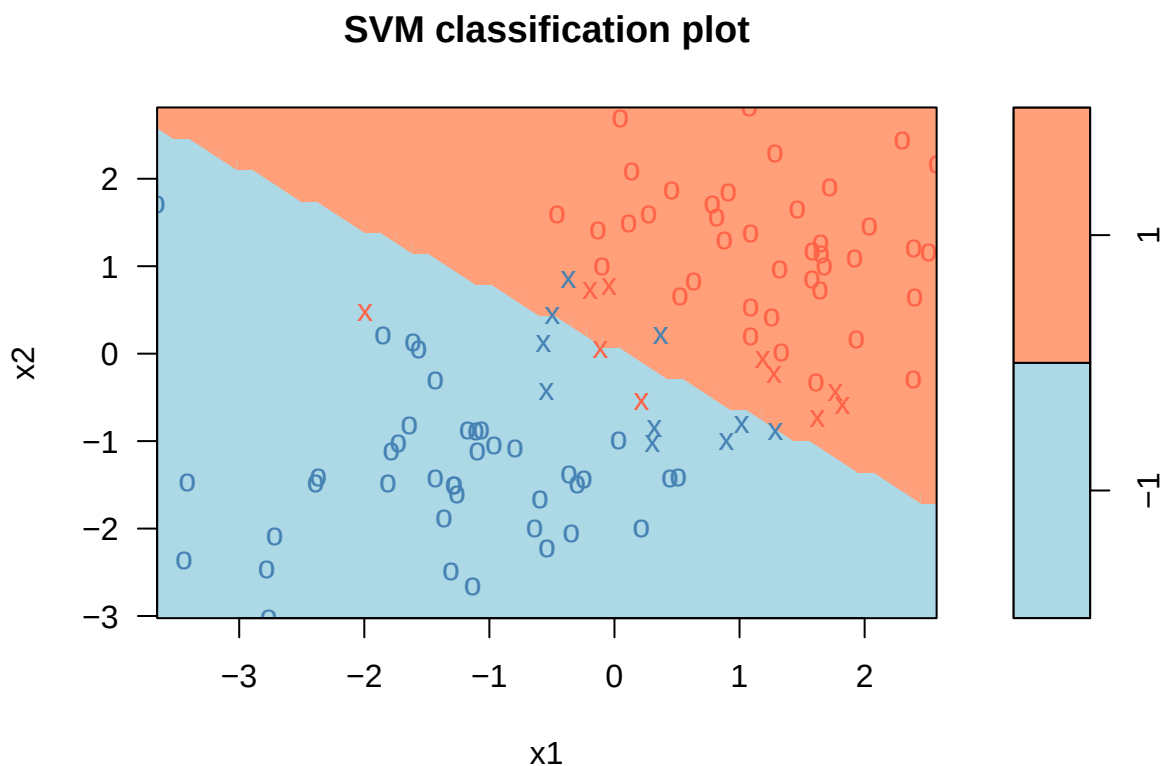


Figura 3: Frontera de decision SVM lineal con margen

Los puntos marcados con $\times$ son los vectores de soporte. Todos los demas puntos ($\circ$) no influyen en la construccion del hiperplano.

1.2. Efecto del parametro C

Vamos a ver la influencia de diferentes valores de C

```

costos <- c(0.01, 0.1, 1, 10, 100)
#Para cada c aplicamos el SVM y calculamos el total nSV
n_sv_vec <- sapply(costos, function(C){
  mod <- svm(y ~ x1 + x2, data = dat_svm,
             kernel = "linear", cost = C, scale = TRUE)
  sum(mod$nSV)
})

df_costo <- data.frame(C = costos, n_SV = n_sv_vec)

# Vectores de soporte por costo :
df_costo

```

```

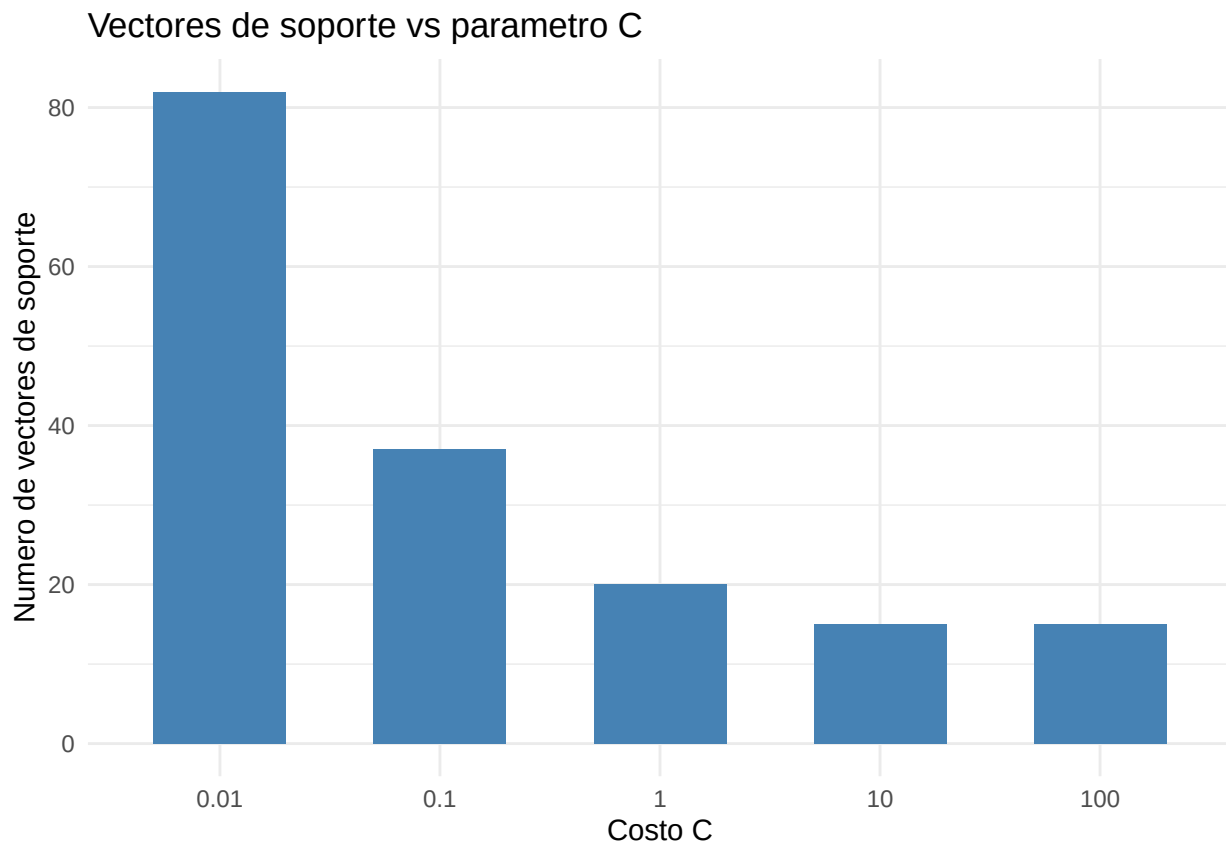
##      C n_SV
## 1 1e-02  82
## 2 1e-01  37
## 3 1e+00  20
## 4 1e+01  15
## 5 1e+02  15

```

```

ggplot(df_costo, aes(x = factor(C), y = n_SV)) +
  geom_col(fill = "steelblue", width = 0.6) +
  labs(title = "Vectores de soporte vs parametro C",
       x = "Costo C", y = "Numero de vectores de soporte") +
  theme_minimal(base_size = 11)

```



```
kable(df_costo, booktabs = TRUE,
      caption = "Numero de vectores de soporte por valor de C")
```

Cuadro 3: Numero de vectores de soporte por valor de C

C	n_SV
1e-02	82
1e-01	37
1e+00	20
1e+01	15
1e+02	15

Conforme C aumenta, el número SV disminuye.

1.3. Seleccin del costo con validacion cruzada: tune

```
set.seed(123)
tune_lin <- tune(svm,
  y ~ x1 + x2,
  data = dat_svm,
  kernel = "linear",
  ranges = list(cost = c(0.01, 0.1, 1, 5, 10)),
  tunecontrol = tune.control(cross = 5))

# Mejor costo encontrado :
tune_lin$best.parameters
```

```
## cost
## 1 0.01
```

```
kable(tune_lin$performances, digits = 4, booktabs = TRUE,
      caption = "Error CV (5-fold) por valor de C - kernel lineal")
```

Cuadro 4: Error CV (5-fold) por valor de C - kernel lineal

cost	error	dispersion
0.01	0.06	0.0548
0.10	0.08	0.0758
1.00	0.08	0.0758
5.00	0.06	0.0548
10.00	0.06	0.0548

```
mod_svm_best <- tune_lin$best.model
mod_svm_best_prob <- svm(y ~ x1 + x2, data = dat_svm,
  kernel = "linear",
  cost = tune_lin$best.parameters$cost,
```

```

scale = TRUE, probability = TRUE)

pred_best_raw <- predict(mod_svm_best_prob, newdata = dat_svm,
                        probability = TRUE)
pi_best      <- attr(pred_best_raw, "probabilities")[,"1"]
pred_best    <- predict(mod_svm_best, newdata = dat_svm)
cm_best      <- confusionMatrix(pred_best, dat_svm$y, positive = "1")

tab_svm_best <- metricas_full(cm_best, pi_best, dat_svm_bin,
                             paste0("SVM Lineal (C=",
                                     tune_lin$best.parameters$cost, ")"))

# Metricas mejor SVM lineal :
tab_svm_best

##                               Metodo Accuracy Sensibilidad Especificidad   F1   AUC
## Accuracy SVM Lineal (C=0.01)      0.93          0.96          0.9 0.932 0.9904

kable(tab_svm_best, row.names = FALSE, booktabs = TRUE,
      caption = "Metricas mejor SVM Lineal (C optimo por CV)")

```

Cuadro 5: Metricas mejor SVM Lineal (C optimo por CV)

Metodo	Accuracy	Sensibilidad	Especificidad	F1	AUC
SVM Lineal (C=0.01)	0.93	0.96	0.9	0.932	0.9904

1.4. SVM con kernel radial (RBF)

Creamos otro conjunto por simulación

```

set.seed(99)
n_c <- 150
r1 <- sqrt(runif(n_c/2, 0, 1)); theta1 <- runif(n_c/2, 0, 2*pi)
r2 <- sqrt(runif(n_c/2, 1.5, 3)); theta2 <- runif(n_c/2, 0, 2*pi)

dat_circ <- data.frame(
  x1 = c(r1*cos(theta1), r2*cos(theta2)),
  x2 = c(r1*sin(theta1), r2*sin(theta2)),
  y = factor(rep(c(1,-1), each = n_c/2))
)
dat_circ_bin <- ifelse(dat_circ$y == "1", 1, 0)
cbind(dat_circ, dat_circ_bin)

```

```

##           x1           x2  y dat_circ_bin
## 1  0.64456689  0.41139444  1           1
## 2 -0.32869608 -0.07576649  1           1
## 3  0.61078975  0.55785359  1           1
## 4 -0.69750803 -0.71133067  1           1
## 5 -0.11380108 -0.72252536  1           1
## 6 -0.92936537 -0.32077107  1           1

```

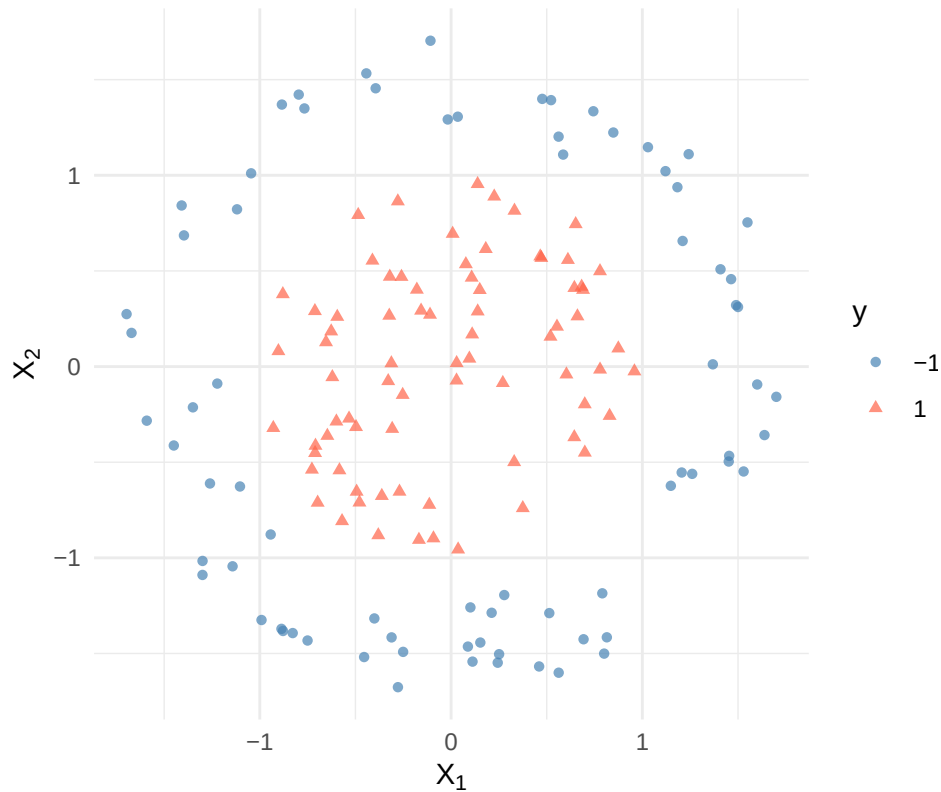
## 7	-0.49379402	-0.65390751	1	1
## 8	0.51969988	0.15649201	1	1
## 9	-0.53343581	-0.27167853	1	1
## 10	-0.32315807	0.26623977	1	1
## 11	0.46626406	0.57568673	1	1
## 12	0.66056107	0.26288928	1	1
## 13	-0.17896251	0.40225476	1	1
## 14	0.68986971	0.40122798	1	1
## 15	0.37491612	-0.73975536	1	1
## 16	0.68252458	0.41755356	1	1
## 17	0.32981722	-0.49910516	1	1
## 18	0.13911686	0.28849870	1	1
## 19	-0.31222332	0.01753626	1	1
## 20	0.14985400	0.40053718	1	1
## 21	0.10765139	0.46509639	1	1
## 22	0.27053499	-0.08564443	1	1
## 23	-0.90280640	0.08098779	1	1
## 24	-0.71192952	0.29029353	1	1
## 25	0.87427447	0.09504304	1	1
## 26	0.55355076	0.20896778	1	1
## 27	0.02840498	-0.07248779	1	1
## 28	-0.09219367	-0.89777868	1	1
## 29	0.02896339	0.01844859	1	1
## 30	-0.30792946	-0.32537614	1	1
## 31	-0.26973924	-0.65371766	1	1
## 32	-0.32049584	0.46987844	1	1
## 33	-0.49703523	-0.31581559	1	1
## 34	0.47163950	0.56836911	1	1
## 35	0.11020780	0.16835756	1	1
## 36	-0.65389697	0.12753826	1	1
## 37	0.69921855	-0.44952630	1	1
## 38	-0.27834039	0.86393168	1	1
## 39	0.77881371	-0.01535933	1	1
## 40	-0.56996362	-0.80782628	1	1
## 41	-0.48552518	0.79269423	1	1
## 42	0.00825189	0.69439428	1	1
## 43	0.33146169	0.81502269	1	1
## 44	0.13878549	0.95450978	1	1
## 45	-0.41126259	0.55423207	1	1
## 46	-0.87948851	0.37867744	1	1
## 47	0.60293768	-0.04093210	1	1
## 48	-0.62679988	0.18409568	1	1
## 49	-0.36212433	-0.67599492	1	1
## 50	-0.15684112	0.29272984	1	1
## 51	-0.71095957	-0.45165154	1	1
## 52	-0.72753624	-0.53839070	1	1
## 53	0.82863770	-0.25740859	1	1
## 54	0.65138905	0.74483678	1	1
## 55	0.07720800	0.53536281	1	1
## 56	0.77845913	0.49870729	1	1
## 57	-0.11018083	0.27130149	1	1
## 58	-0.64682706	-0.36183665	1	1
## 59	0.95845650	-0.02474337	1	1
## 60	-0.62078030	-0.05504158	1	1

## 61	0.03676050	-0.95591886	1	1
## 62	-0.25931547	0.46915898	1	1
## 63	-0.70892162	-0.41372383	1	1
## 64	-0.47908397	-0.71042177	1	1
## 65	-0.59954730	-0.28703266	1	1
## 66	-0.16813040	-0.90557529	1	1
## 67	0.64407907	-0.36895275	1	1
## 68	-0.38050534	-0.88146328	1	1
## 69	0.09555336	0.04176262	1	1
## 70	0.22555345	0.88908145	1	1
## 71	-0.25306040	-0.14778587	1	1
## 72	-0.58373978	-0.54188299	1	1
## 73	-0.59493044	0.26035344	1	1
## 74	0.18134967	0.61485745	1	1
## 75	0.69884792	-0.19685760	1	1
## 76	0.74364193	1.33467163	-1	0
## 77	1.36835143	0.01165685	-1	0
## 78	-0.79642772	1.42175610	-1	0
## 79	1.24209296	1.11033888	-1	0
## 80	-1.29957904	-1.08929594	-1	0
## 81	0.81422494	-1.41543509	-1	0
## 82	-0.40118111	-1.31674643	-1	0
## 83	1.12085813	1.02154372	-1	0
## 84	0.11208182	-1.54235833	-1	0
## 85	0.47656702	1.39943800	-1	0
## 86	-0.39429427	1.45479757	-1	0
## 87	-0.88465486	1.36964055	-1	0
## 88	-1.39646029	0.68570444	-1	0
## 89	1.48987786	0.32119697	-1	0
## 90	0.21204669	-1.28708601	-1	0
## 91	-0.27787689	-1.67604760	-1	0
## 92	-1.29930479	-1.01617332	-1	0
## 93	-0.99105501	-1.32492187	-1	0
## 94	-1.67062312	0.17594343	-1	0
## 95	0.58589548	1.10820238	-1	0
## 96	0.24377432	-1.54805597	-1	0
## 97	1.46386120	0.45692403	-1	0
## 98	-0.25071451	-1.49158351	-1	0
## 99	-1.22227082	-0.08892343	-1	0
## 100	0.56253369	-1.60035212	-1	0
## 101	0.27800016	-1.19442396	-1	0
## 102	1.40850572	0.50855376	-1	0
## 103	-1.34969280	-0.21308796	-1	0
## 104	-0.82805824	-1.39320750	-1	0
## 105	1.14882208	-0.62332676	-1	0
## 106	0.84814035	1.22355087	-1	0
## 107	1.26046004	-0.56078307	-1	0
## 108	1.45368929	-0.46681669	-1	0
## 109	1.18317689	0.93768894	-1	0
## 110	-1.69647413	0.27439911	-1	0
## 111	-0.88676988	-1.37118240	-1	0
## 112	1.63821575	-0.35810888	-1	0
## 113	-1.40868571	0.84213074	-1	0
## 114	0.56198060	1.20191361	-1	0

```
## 115 0.46027095 -1.56777350 -1 0
## 116 0.52264436 1.39277885 -1 0
## 117 0.69251048 -1.42560475 -1 0
## 118 0.79050808 -1.18542574 -1 0
## 119 -0.76685590 1.34953983 -1 0
## 120 1.02921521 1.14685794 -1 0
## 121 -1.59101271 -0.28288715 -1 0
## 122 1.52931901 -0.54844298 -1 0
## 123 0.80015635 -1.50061013 -1 0
## 124 -0.75039514 -1.43218331 -1 0
## 125 -0.87872113 -1.38227988 -1 0
## 126 0.10104634 -1.25905559 -1 0
## 127 1.49998558 0.31140461 -1 0
## 128 -1.26044144 -0.61120382 -1 0
## 129 -1.44960035 -0.41269001 -1 0
## 130 -0.01749425 1.29170018 -1 0
## 131 -0.45467355 -1.51856323 -1 0
## 132 1.21045673 0.65664270 -1 0
## 133 1.45150515 -0.49646065 -1 0
## 134 -1.10357714 -0.62684786 -1 0
## 135 -0.94316617 -0.87781260 -1 0
## 136 0.08793217 -1.46384751 -1 0
## 137 0.25096516 -1.50358418 -1 0
## 138 1.70026295 -0.15826613 -1 0
## 139 -0.44307419 1.53266553 -1 0
## 140 -1.04531642 1.01028420 -1 0
## 141 0.03494538 1.30623137 -1 0
## 142 0.51341210 -1.28887661 -1 0
## 143 -1.11908812 0.82205707 -1 0
## 144 1.60031297 -0.09373167 -1 0
## 145 -1.14242128 -1.04421664 -1 0
## 146 0.15288496 -1.44274641 -1 0
## 147 -0.10780692 1.70344668 -1 0
## 148 1.54944391 0.75370582 -1 0
## 149 1.20547812 -0.55379585 -1 0
## 150 -0.31109607 -1.41579052 -1 0
```

```
ggplot(dat_circ, aes(x1, x2, color = y, shape = y)) +
  geom_point(size = 1.5, alpha = 0.7) +
  scale_color_manual(values = c("steelblue", "tomato")) +
  coord_equal() +
  labs(title = "Clases con patron circular",
       x = expression(X[1]), y = expression(X[2])) +
  theme_minimal(base_size = 11)
```

Clases con patron circular



Para estos datos, de acuerdo con la figura, proponemos un kernel radial para que pueda separar los puntos de forma adecuada

```
set.seed(123)
tune_rbf <- tune(svm,
  y ~ x1 + x2,
  data = dat_circ,
  kernel = "radial",
  ranges = list(cost = c(0.1, 1, 10),
    gamma = c(0.1, 0.5, 1, 2)),
  tunecontrol = tune.control(cross = 5))
```

Mejores hiperparametros :

```
tune_rbf$best.parameters
```

```
## cost gamma
## 3 10 0.1
```

```
kable(tune_rbf$performances, digits = 4, booktabs = TRUE,
  caption = "Error CV (5-fold) - kernel radial")
```

Cuadro 6: Error CV (5-fold) - kernel radial

cost	gamma	error	dispersion
0.1	0.1	0.5200	0.1709

cost	gamma	error	dispersion
1.0	0.1	0.0067	0.0149
10.0	0.1	0.0000	0.0000
0.1	0.5	0.0267	0.0435
1.0	0.5	0.0000	0.0000
10.0	0.5	0.0000	0.0000
0.1	1.0	0.0400	0.0548
1.0	1.0	0.0000	0.0000
10.0	1.0	0.0067	0.0149
0.1	2.0	0.0400	0.0435
1.0	2.0	0.0000	0.0000
10.0	2.0	0.0067	0.0149

```
mod_rbf_best <- tune_rbf$best.model
mod_rbf_prob <- svm(y ~ x1 + x2, data = dat_circ,
  kernel = "radial",
  cost = tune_rbf$best.parameters$cost,
  gamma = tune_rbf$best.parameters$gamma,
  scale = TRUE, probability = TRUE)

pred_rbf_raw <- predict(mod_rbf_prob, newdata = dat_circ, probability = TRUE)
pi_rbf_circ <- attr(pred_rbf_raw, "probabilities")[,"1"]
pred_rbf <- predict(mod_rbf_best, newdata = dat_circ)
cm_rbf <- confusionMatrix(pred_rbf, dat_circ$y, positive = "1")

tab_rbf_circ <- metrics_full(cm_rbf, pi_rbf_circ, dat_circ_bin,
  paste0("SVM Radial (C=",
    tune_rbf$best.parameters$cost,
    ", g=", tune_rbf$best.parameters$gamma, ")"))

# Metricas SVM Radial (datos circulares) :
tab_rbf_circ
```

```
##                               Metodo Accuracy Sensibilidad Especificidad F1 AUC
## Accuracy SVM Radial (C=10, g=0.1)          1              1              1  1  1
```

```
kable(tab_rbf_circ, row.names = FALSE, booktabs = TRUE,
  caption = "Metricas SVM Radial optimo - datos circulares")
```

Cuadro 7: Metricas SVM Radial optimo - datos circulares

Metodo	Accuracy	Sensibilidad	Especificidad	F1	AUC
SVM Radial (C=10, g=0.1)	1	1	1	1	1

```
plot(mod_rbf_best, dat_circ, x2 ~ x1,
  col = c("lightblue", "lightsalmon"),
  symbolPalette = c("steelblue", "tomato"))
```

SVM classification plot

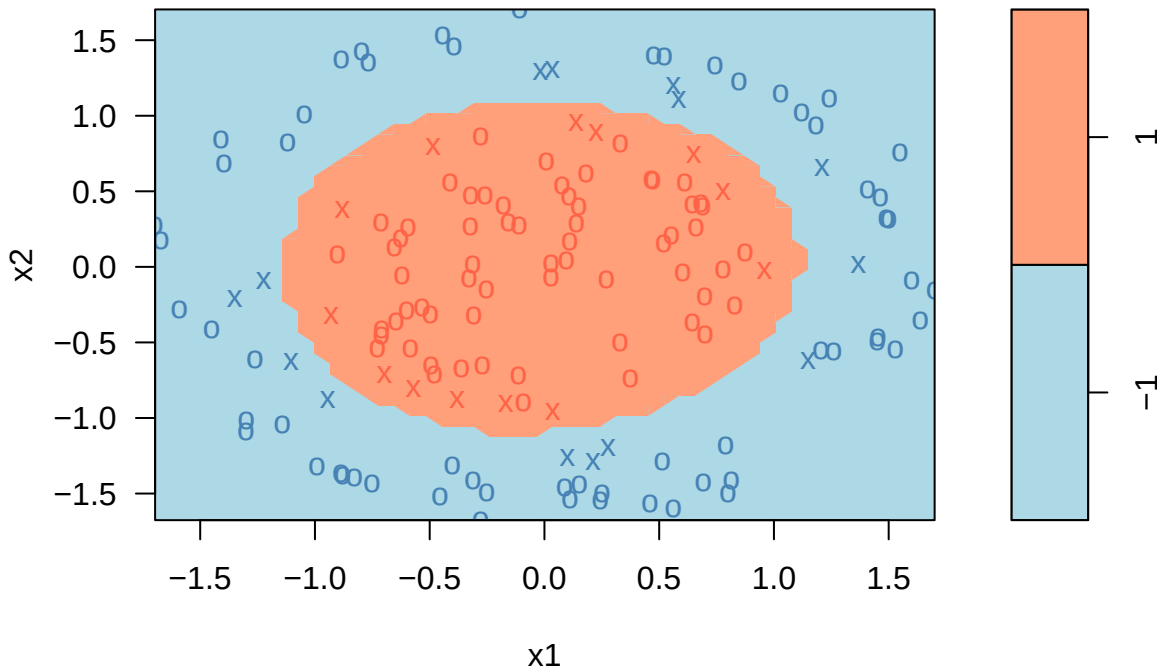


Figura 4: Frontera de decision SVM con kernel radial

1.5. SVM con datos reales: Default (ISLR2)

Usamos los datos de default que trabajamos en otros casos

Creamos una función para aplicar validación cruzada para los datos de entrenamiento

```
ctrl_svm <- trainControl(method = "cv", number = 5,
  classProbs = TRUE,
  summaryFunction = twoClassSummary)
```

Usamos los datos de entrenamiento para “entrenar” el método SVM

```
data(Default, package = "ISLR2")
Default <- Default %>% mutate(default_bin = ifelse(default == "Yes", 1, 0))

set.seed(123)
n_def <- nrow(Default)
idx <- sample(1:n_def, floor(0.7 * n_def))
train_d <- Default[idx, ]
test_d <- Default[-idx, ]
y_test <- test_d$default
y_test_bin <- test_d$default_bin
```

```
set.seed(123)
mod_svm_def <- train(default ~ balance + income,
  data = train_d,
```

```

method      = "svmRadial",
trControl   = ctrl_svm,
metric      = "ROC",
preProcess  = c("center","scale"))

```

Mejores hiperparametros :

```
mod_svm_def$bestTune
```

```
##      sigma C
```

```
## 3 1.240798 1
```

Hacemos las predicciones para los datos de test

```

pi_svm_def  <- predict(mod_svm_def, newdata = test_d, type = "prob")[, "Yes"]
pred_svm_def <- factor(ifelse(pi_svm_def >= 0.5, "Yes", "No"),
                      levels = c("No", "Yes"))
#creamos la matriz de confusi3n
cm_svm_def  <- confusionMatrix(pred_svm_def, test_d$default, positive = "Yes")
# Metricas SVM Radial - Default (test 30%) :
tab_svm_def <- metricas_full(cm_svm_def, pi_svm_def, y_test_bin,
                             "SVM Radial sobre Default")

kable(tab_svm_def, row.names = FALSE, booktabs = TRUE,
      caption = "Metricas SVM Radial sobre Default (test 30\\%)")

```

Cuadro 8: Metricas SVM Radial sobre Default (test 30 %)

Metodo	Accuracy	Sensibilidad	Especificidad	F1	AUC
SVM Radial sobre Default	0.972	0.24	0.9972	0.3636	0.6926

```

kable(cm_svm_def$table,
      caption = "Matriz de Confusion - SVM Radial sobre Default (test 30\\%)")

```

Cuadro 9: Matriz de Confusion - SVM Radial sobre Default (test 30 %)

	No	Yes
No	2892	76
Yes	8	24

2. Clasificacion con Clases Desbalanceadas

El conjunto `Default` tiene solo $\approx 3,3\%$ de casos positivos (`Yes`). Se aplican las tecnicas presentadas en clases y se comparan con F1 y AUC para capturar el desempeno en la clase minoritaria.

2.1. Exploracion del desbalance

```
Default %>%  
  count(default) %>%  
  mutate(proporcion = round(n / sum(n), 3)) %>%  
  kable(caption = "Distribucion de la variable respuesta", booktabs = TRUE)
```

Cuadro 10: Distribucion de la variable respuesta

default	n	proporcion
No	9667	0.967
Yes	333	0.033

2.2. Modelo base sin balancear

```
mod_base <- glm(default ~ balance + income,  
  data = train_d,  
  family = binomial())  
  
pi_base <- predict(mod_base, newdata = test_d, type = "response")  
pred_base <- factor(ifelse(pi_base >= 0.5, "Yes", "No"), levels = c("No", "Yes"))  
cm_base <- confusionMatrix(pred_base, y_test, positive = "Yes")  
  
tab_base <- metricas_full(cm_base, pi_base, y_test_bin, "Sin balanceo (p=0.5)")  
  
# Metricas modelo sin balancear :  
tab_base  
  
##  
##          Metodo Accuracy Sensibilidad Especificidad      F1      AUC  
## Accuracy Sin balanceo (p=0.5)    0.9737          0.31      0.9966 0.4397 0.9473  
  
kable(cm_base$table,  
  caption = "Modelo base (sin balancear) - Regresion Logistica")
```

Cuadro 11: Modelo base (sin balancear) - Regresion Logistica

	No	Yes
No	2890	69
Yes	10	31

2.3. Undersampling aleatorio

```
set.seed(123)
train_yes <- train_d[train_d$default == "Yes", ]
train_no  <- train_d[train_d$default == "No", ]

idx_under <- sample(1:nrow(train_no), nrow(train_yes))
train_under <- rbind(train_yes, train_no[idx_under, ])

# Distribucion tras undersampling :
table(train_under$default)

##
## No Yes
## 233 233

mod_under <- glm(default ~ balance + income,
                  data = train_under,
                  family = binomial())

pi_under <- predict(mod_under, newdata = test_d, type = "response")
pred_under <- factor(ifelse(pi_under >= 0.5, "Yes", "No"), levels = c("No", "Yes"))
cm_under <- confusionMatrix(pred_under, y_test, positive = "Yes")

tab_under <- metricas_full(cm_under, pi_under, y_test_bin,
                           "Undersampling aleatorio")

# Metricas undersampling :
#tab_under

kable(cm_under$table, caption = "Modelo con Undersampling aleatorio")
```

Cuadro 12: Modelo con Undersampling aleatorio

	No	Yes
No	2441	11
Yes	459	89

2.4. Oversampling aleatorio

```
set.seed(123)
n_no <- nrow(train_no)
idx_ov <- sample(1:nrow(train_yes), n_no, replace = TRUE)
train_over <- rbind(train_no, train_yes[idx_ov, ])

# Distribucion tras oversampling :
table(train_over$default)
```

```
##
##   No   Yes
## 6767 6767

mod_over <- glm(default ~ balance + income,
  data      = train_over,
  family    = binomial())

pi_over  <- predict(mod_over, newdata = test_d, type = "response")
pred_over <- factor(ifelse(pi_over >= 0.5, "Yes", "No"), levels = c("No", "Yes"))
cm_over   <- confusionMatrix(pred_over, y_test, positive = "Yes")

tab_over <- metricas_full(cm_over, pi_over, y_test_bin, "Oversampling aleatorio")

# Metricas oversampling :
#tab_over

kable(cm_over$table, caption = "Modelo con Oversampling aleatorio")
```

Cuadro 13: Modelo con Oversampling aleatorio

	No	Yes
No	2504	14
Yes	396	86

2.5. SMOTE

SMOTE crea ejemplos sintéticos de la clase minoritaria:

$$S_i^j = X_j + \lambda(X_i^j - X_j), \quad \lambda \sim U(0, 1)$$

```
ctrl_smote <- trainControl(
  method      = "cv", number = 5,
  classProbs  = TRUE,
  summaryFunction = twoClassSummary,
  sampling     = "smote"
)

set.seed(123)
mod_smote <- train(default ~ balance + income,
  data      = train_d,
  method    = "glm",
  family    = binomial(),
  trControl = ctrl_smote,
  metric    = "ROC")

pi_smote  <- predict(mod_smote, newdata = test_d, type = "prob")[, "Yes"]
pred_smote <- factor(ifelse(pi_smote >= 0.5, "Yes", "No"), levels = c("No", "Yes"))
cm_smote   <- confusionMatrix(pred_smote, y_test, positive = "Yes")

tab_smote <- metricas_full(cm_smote, pi_smote, y_test_bin, "SMOTE")
```

```
# Métricas SMOTE :
#tab_smote

kable(cm_smote$table, caption = "Modelo con SMOTE - Regresion Logistica")
```

Cuadro 14: Modelo con SMOTE - Regresion Logistica

	No	Yes
No	2546	16
Yes	354	84

2.6. Probabilidad de corte optima

Se busca el corte que iguala la sensibilidad y la especificidad:

```
cortes <- seq(0.01, 0.99, by = 0.01)

metricas_corte <- sapply(cortes, function(c){
  pred_c <- factor(ifelse(pi_base >= c, "Yes", "No"), levels = c("No", "Yes"))
  cm_c <- confusionMatrix(pred_c, y_test, positive = "Yes")
  c(Sens = as.numeric(cm_c$byClass["Sensitivity"]),
    Spec = as.numeric(cm_c$byClass["Specificity"]))
})

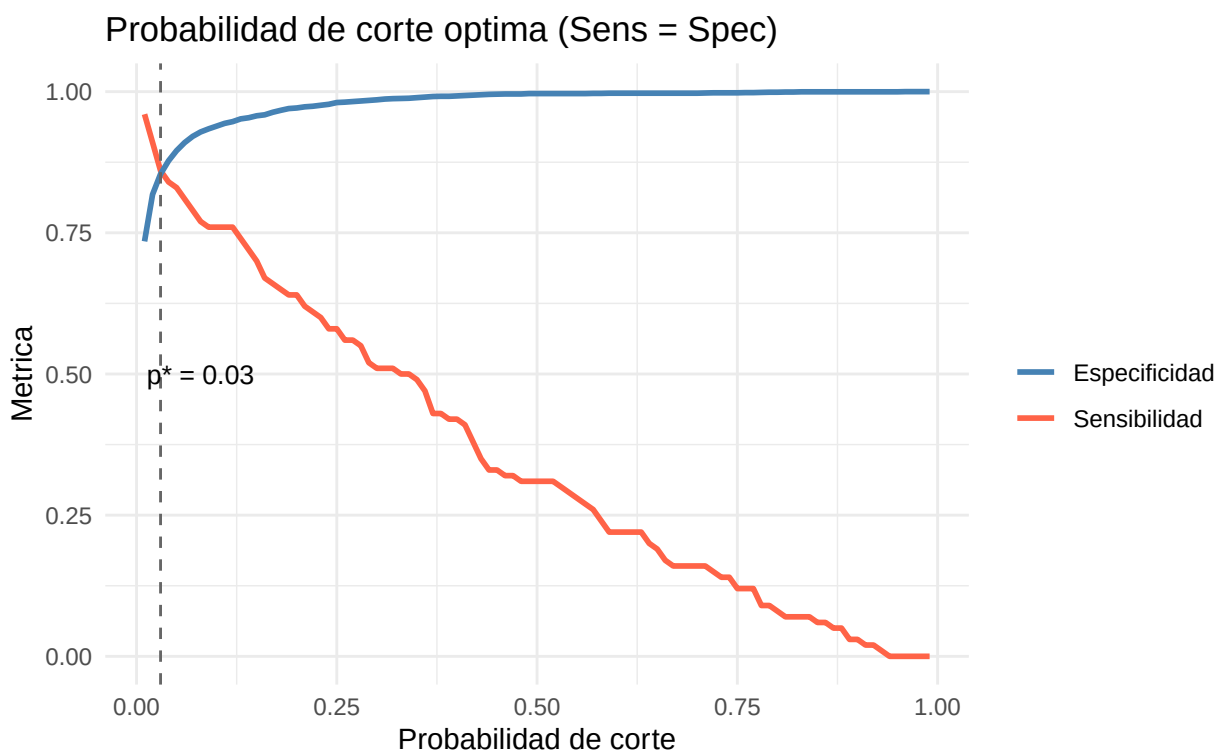
df_corte <- data.frame(
  corte = cortes,
  Sens = metricas_corte["Sens",],
  Spec = metricas_corte["Spec",]
)

dif <- abs(df_corte$Sens - df_corte$Spec)
corte_opt <- df_corte$corte[which.min(dif)]

# Probabilidad de corte optima (Sens = Spec):
corte_opt
```

```
## [1] 0.03
```

```
ggplot(df_corte, aes(x = corte)) +
  geom_line(aes(y = Sens, color = "Sensibilidad"), linewidth = 1) +
  geom_line(aes(y = Spec, color = "Especificidad"), linewidth = 1) +
  geom_vline(xintercept = corte_opt, linetype = "dashed", color = "gray40") +
  annotate("text", x = corte_opt + 0.05, y = 0.5,
    label = paste0("p* = ", corte_opt), size = 3.5) +
  scale_color_manual(values = c("Sensibilidad" = "tomato",
    "Especificidad" = "steelblue")) +
  labs(title = "Probabilidad de corte optima (Sens = Spec)",
    x = "Probabilidad de corte", y = "Metrica", color = "") +
  theme_minimal(base_size = 11)
```



```
pred_opt <- factor(ifelse(pi_base >= corte_opt, "Yes", "No"),
  levels = c("No", "Yes"))
cm_opt <- confusionMatrix(pred_opt, y_test, positive = "Yes")

tab_opt <- metricas_full(cm_opt, pi_base, y_test_bin,
  paste0("Corte optimo (p=", corte_opt, ")"))

# Metricas con corte optimo :
#tab_opt

kable(cm_opt$table,
  caption = paste0("Modelo base con corte optimo (p = ", corte_opt, ")"))
```

Cuadro 15: Modelo base con corte optimo (p = 0.03)

	No	Yes
No	2477	14
Yes	423	86

2.7. Resumen comparativo de tecnicas

```
resumen_db <- bind_rows(tab_base, tab_under, tab_over, tab_smote, tab_opt)
```

```
# Comparacion de tecnicas para clases desbalanceadas :
#resumen_db

kable(resumen_db, row.names = FALSE, booktabs = TRUE,
      caption = "Comparacion de tecnicas para clases desbalanceadas - Default (test 30\\%)")
```

Cuadro 16: Comparacion de tecnicas para clases desbalanceadas - Default (test 30 %)

Metodo	Accuracy	Sensibilidad	Especificidad	F1	AUC
Sin balanceo (p=0.5)	0.9737	0.31	0.9966	0.4397	0.9473
Undersampling aleatorio	0.8433	0.89	0.8417	0.2747	0.9473
Oversampling aleatorio	0.8633	0.86	0.8634	0.2955	0.9473
SMOTE	0.8767	0.84	0.8779	0.3123	0.9472
Corte optimo (p=0.03)	0.8543	0.86	0.8541	0.2824	0.9473

Las tecnicas de balanceo mejoran significativamente la sensibilidad y el F1 Score a costa de reducir la especificidad. El AUC es identico para el modelo base y el corte optimo (mismo modelo, distinto umbral), mientras que los metodos de balanceo pueden modificar el AUC al cambiar el modelo entrenado.

3. k-Vecinos Mas Cercanos (k-NN)

El k-NN es un clasificador no parametrico que asigna a una nueva observacion la clase mas frecuente (moda) entre sus k vecinos mas cercanos en el espacio de predictores.

3.1. Ejemplo manual

Se considera la muestra presentada en clases ($n = 10$, predictores continuos x_1, x_2 , respuesta $y \in \{a, b\}$):

```
options(scipen = 999)
datos <- data.frame(
  y = factor(c("a", "a", "a", "a", "a", "b", "b", "b", "b", "b")),
  x1 = c(9, 6, 1, 3, 3, 1, 9, 9, 4, 6),
  x2 = c(6, 2, 6, 9, 7, 9, 8, 2, 6, 4)
)
```

```
# Datos de entrenamiento :
datos
```

```
##      y x1 x2
## 1   a  9  6
## 2   a  6  2
## 3   a  1  6
## 4   a  3  9
## 5   a  3  7
## 6   b  1  9
## 7   b  9  8
## 8   b  9  2
## 9   b  4  6
## 10  b  6  4
```

```
kable(t(datos), caption = "Muestra de entrenamiento")
```

Cuadro 17: Muestra de entrenamiento

y	a	a	a	a	a	b	b	b	b	b
x1	9	6	1	3	3	1	9	9	4	6
x2	6	2	6	9	7	9	8	2	6	4

Se clasifica la nueva observacion $\mathbf{x} = (5, 2)$ calculando la distancia euclidiana al cuadrado $d^2(\mathbf{x}_i, \mathbf{x})$ para cada punto:

```
xnew <- c(5, 2)

d2 <- (datos$x1 - xnew[1])^2 + (datos$x2 - xnew[2])^2

tabla_dist <- data.frame(
  i = 1:10, y = as.character(datos$y),
  x1 = datos$x1, x2 = datos$x2, d2 = d2
)
```

```
# Tabla de distancias ordenadas :
tabla_dist[order(tabla_dist$d2), ]
```

```
##      i y x1 x2 d2
## 2    2 a  6  2  1
## 10  10 b  6  4  5
## 8    8 b  9  2 16
## 9    9 b  4  6 17
## 5    5 a  3  7 29
## 1    1 a  9  6 32
## 3    3 a  1  6 32
## 7    7 b  9  8 52
## 4    4 a  3  9 53
## 6    6 b  1  9 65
```

```
kable(tabla_dist[order(tabla_dist$d2), ], digits = 0, row.names = FALSE,
      caption = "Distancias al cuadrado desde (5,2), ordenadas de menor a mayor")
```

Cuadro 18: Distancias al cuadrado desde (5,2), ordenadas de menor a mayor

i	y	x1	x2	d2
2	a	6	2	1
10	b	6	4	5
8	b	9	2	16
9	b	4	6	17
5	a	3	7	29
1	a	9	6	32
3	a	1	6	32
7	b	9	8	52
4	a	3	9	53
6	b	1	9	65

```
tabla_ord <- tabla_dist[order(tabla_dist$d2), ]
```

```
# 1-NN
N1 <- tabla_ord$y[1]
cat("1-NN: vecino =", N1, "=> Yhat =", N1, "\n")
```

```
## 1-NN: vecino = a => Yhat = a
```

```
# 2-NN
N2 <- tabla_ord$y[1:2]
cat("2-NN: vecinos =", paste(N2, collapse=" "),
    "=> Yhat =", names(sort(table(N2), decreasing = TRUE))[1], "\n")
```

```
## 2-NN: vecinos = a, b => Yhat = a
```

```
# 3-NN
N3 <- tabla_ord$y[1:3]
cat("3-NN: vecinos =", paste(N3, collapse=" "),
    "=> Yhat =", names(sort(table(N3), decreasing = TRUE))[1], "\n")
```

```
## 3-NN: vecinos = a, b, b => Yhat = b
```

3.2. Ajuste con `class::knn`

```
train_X <- datos[, c("x1", "x2")]
test_X <- data.frame(x1 = 5, x2 = 2)

pred_k1 <- knn(train = train_X, test = test_X, cl = datos$y, k = 1)
cat("k=1:", as.character(pred_k1), "\n")
```

```
## k=1: a
```

```
pred_k2 <- knn(train = train_X, test = test_X, cl = datos$y, k = 2)
cat("k=2:", as.character(pred_k2), "\n")
```

```
## k=2: a
```

```
pred_k3 <- knn(train = train_X, test = test_X, cl = datos$y, k = 3)
cat("k=3:", as.character(pred_k3), "\n")
```

```
## k=3: b
```

3.3. Selección de `k` con datos reales: Default (ISLR2)

Aplicamos k-NN al conjunto Default para predecir impago en función de `balance` e `income`, estandarizando previamente los predictores. Se usan `caret` y `PROC` para obtener probabilidades de clase y calcular AUC:

```
data(Default, package = "ISLR2")
Default <- Default %>% mutate(default_bin = ifelse(default == "Yes", 1, 0))

set.seed(123)
n_def <- nrow(Default)
idx <- sample(1:n_def, floor(0.7 * n_def))
train_d <- Default[idx, ]
test_d <- Default[-idx, ]
y_test <- test_d$default
y_test_bin <- test_d$default_bin

# Estandarización con parámetros de entrenamiento
pre_sc <- preProcess(train_d[, c("balance", "income")],
                      method = c("center", "scale"))

train_sc <- train_d
test_sc <- test_d
```

```

train_sc[, c("balance_sc", "income_sc")] <-
  predict(pre_sc, train_d[, c("balance", "income")])
test_sc[, c("balance_sc", "income_sc")] <-
  predict(pre_sc, test_d[, c("balance", "income")])

# Ajuste con caret para obtener probabilidades (necesarias para AUC)
ctrl_knn <- trainControl(method = "cv", number = 5,
  classProbs = TRUE,
  summaryFunction = twoClassSummary)

ks <- c(1, 5, 15, 30)

res_knn <- lapply(ks, function(k){
  set.seed(123)
  mod_k <- train(default ~ balance_sc + income_sc,
    data = train_sc,
    method = "knn",
    trControl = trainControl(classProbs = TRUE),
    tuneGrid = data.frame(k = k))
  pi_k <- predict(mod_k, newdata = test_sc, type = "prob")[, "Yes"]
  pred_k <- factor(ifelse(pi_k >= 0.5, "Yes", "No"), levels = c("No", "Yes"))
  cm_k <- confusionMatrix(pred_k, y_test, positive = "Yes")
  metricas_full(cm_k, pi_k, y_test_bin, paste0("k = ", k))
}) %>% bind_rows()

# Métricas k-NN por valor de k (test 30%) :
res_knn

##           Metodo Accuracy Sensibilidad Especificidad      F1      AUC
## Accuracy...1 k = 1    0.9537          0.31          0.9759 0.3085 0.6429
## Accuracy...2 k = 5    0.9700          0.32          0.9924 0.4156 0.7972
## Accuracy...3 k = 15   0.9723          0.30          0.9955 0.4196 0.8721
## Accuracy...4 k = 30   0.9730          0.29          0.9966 0.4173 0.8961

kable(res_knn, row.names = FALSE, booktabs = TRUE,
  caption = "k-NN sobre Default: metricas por valor de k (test 30\\%)")

```

Cuadro 19: k-NN sobre Default: metricas por valor de k (test 30 %)

Metodo	Accuracy	Sensibilidad	Especificidad	F1	AUC
k = 1	0.9537	0.31	0.9759	0.3085	0.6429
k = 5	0.9700	0.32	0.9924	0.4156	0.7972
k = 15	0.9723	0.30	0.9955	0.4196	0.8721
k = 30	0.9730	0.29	0.9966	0.4173	0.8961

4. Ejercicio Propuesto

Ejercicio Considere la base de datos OJ de ISLR2, que contiene informacion sobre 1070 compras de jugo de naranja. La variable respuesta es **Purchase** (CH = Citrus Hill, MM = Minute Maid) en funcion de predictores como **LoyalCH**, **SalePriceMM**, **SalePriceCH**, **PriceDiff**.

- a) Divida la muestra en 70 % entrenamiento y 30 % prueba. Ajuste un **k-NN** con $k \in \{1, 3, 5, 7, 15\}$ y seleccione el mejor k usando AUC en el conjunto de prueba. Reporte Accuracy, Sensibilidad, Especificidad, F1 y AUC para cada k .
- b) Ajuste una **SVM con kernel lineal** y una **SVM con kernel radial** usando validacion cruzada 5-fold para seleccionar C (y γ en el caso radial). Compare ambas SVM reportando Accuracy, Sensibilidad, Especificidad, F1 y AUC en el conjunto de prueba.
- c) Verifique si existe desbalance en la variable **Purchase**. Si lo hay, aplique al menos dos tecnicas de balanceo y compare el F1 Score y AUC del mejor SVM del literal (b) con y sin balanceo.
- d) Para el modelo SVM radial, calcule la probabilidad de corte optima usando el criterio de igualar sensibilidad y especificidad. Compare las metricas completas (incluyendo F1 y AUC) con el umbral 0.5.

Referencias

- [Hastie et al.(2008)] Trevor Hastie, Robert Tibshirani and Jerome Friedman (2008) *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer Series in Statistics.
- [James et al.(2021)] Gareth James, Daniela Witten, Trevor Hastie and Robert Tibshirani (2021) *An Introduction to Statistical Learning with Applications in R*, 2nd edition. Springer Series in Statistics.